

Resolução da Lista 1 - Métodos numéricos

Engenharia de Computação

18/09/2026

Questão 1: Padrão IEEE-754 (64 bits)

A representação em ponto flutuante no padrão IEEE-754 de dupla precisão é composta por 1 bit de sinal, 11 bits de expoente e 52 bits de fração (mantissa).

Código Python

```
def resolver_ieee754(entrada_binaria):  
    str_limpa = entrada_binaria.replace("_", "").replace("\n", "")  
    str_64bits = str_limpa.ljust(64, '0')  
  
    bit_sinal = str_64bits[0]  
    bits_expoente = str_64bits[1:12]  
    bits_mantissa = str_64bits[12:64]  
  
    s = int(bit_sinal)  
  
    e = 0  
    for i, bit in enumerate(reversed(bits_expoente)):  
        if bit == '1':  
            e += 2**i  
  
    f = 0.0  
    for i, bit in enumerate(bits_mantissa):  
        if bit == '1':  
            f += 2**(-(i + 1))  
  
    forma_matematica = f"{(-1)**{bit_sinal}}*(1.{bits_mantissa})_2^{e-1023}"  
  
    valor_mantissa = 1.0 + f  
    multiplicador = 2**(e - 1023)  
    sinal_mult = -1 if s == 1 else 1  
    valor_decimal = sinal_mult * valor_mantissa * multiplicador  
  
    print(f"Sinal_{s}: {bit_sinal} → {'s' if s==1 else ''}")  
    print(f"Expoente_{e}: {bits_expoente} → {e}")  
    print(f"Fração/Mantissa: {bits_mantissa}")  
    print(f"Formamatemática: {forma_matematica}")  
    print(f"ValorDecimal: {valor_decimal}")  
    print("- " * 70)  
  
entradas = [  
    "0_10000000101_01101001000000000000000000000000000000000000000000000000",  
    "1_10000000101_01101001000000000000000000000000000000000000000000000000",  
    "0_10000000000_00110101000000000000000000000000000000000000000000000000",  
    "0_10000000000_00110101000000000000000000000000000000000000000000000001",  
]  
  
for letra, sequencia in zip(['a', 'b', 'c', 'd'], entradas):  
    print(f"\nItem_{letra}")  
    resolver_ieee754(sequencia)
```

Saída do Terminal

Item a)

[illegible][illegible][illegible]

Logo,

$$(1.f)_2 = 1,41015625, \quad 2^{e-1023} = 2^{1029-1023} = 2^6 = 64,$$

e o valor decimal é $(+1) \times 1,41015625 \times 64 = 90,25$, confirmando a saída do código. O item (b) é idêntico, exceto pelo sinal ($s = 1$), resultando em $-90,25$.

Nos itens (c) e (d), o expoente é $e = 10000000000_2 = 1024$, de modo que $2^{e-1023} = 2^1 = 2$, e a mantissa é $f = 0011010100\dots 0$, cuja soma de potências vale

$$f = 2^{-3} + 2^{-4} + 2^{-6} + 2^{-8} = 0,125 + 0,0625 + 0,015625 + 0,00390625 = 0,20703125,$$

o que dá $(1.f)_2 = 1,20703125$ e valor decimal $1,20703125 \times 2 = 2,4140625$ para o item (c), exatamente como retornado pelo código.

O item (d) usa a mesma mantissa do item (c), porém com o último bit (bit 52, de peso 2^{-52}) igual a 1 em vez de 0. Essa é a menor variação possível representável nesse formato — uma unidade na última posição, ou 1 ULP (*unit in the last place*). O acréscimo à fração é

$$\Delta f = 2^{-52},$$

que, multiplicado pelo fator $2^{e-1023} = 2$, produz uma diferença absoluta no valor decimal de

$$2 \times 2^{-52} = 2^{-51} \approx 4,44 \times 10^{-16}.$$

Isso explica por que os valores decimais dos itens (c) e (d) — 2,4140625 e 2,4140625000000004 — diferem apenas na 16ª casa decimal: essa é justamente a precisão limite representável em ponto flutuante de dupla precisão, e o item (d) ilustra numericamente esse limite.

Questão 2: Polinômio de Maclaurin e Aritmética Decimal

Considere a função

$$f(x) = \cos(x),$$

utilizando aritmética decimal com $k = 4$ algarismos significativos e arredondamento após cada operação.

Código Python

```
1 from decimal import Decimal, localcontext, ROUND_HALF_EVEN
2
3 K4 = 4
4 ALTA = 40
5
6
7 class ContaDecimal:
8     """
9     Wrapper em torno de Decimal que conta quantas operações
10    aritméticas (multiplicação, soma, subtração) são realizadas.
11    Isso evita "chutar" a quantidade de operações: o número
12    reportado é sempre uma consequência direta do código executado.
13    """
14    contador = 0
15
16    def __init__(self, valor):
17        self.valor = valor if isinstance(valor, Decimal) else Decimal(valor)
18
19    def _bin_op(self, outro, func):
20        ContaDecimal.contador += 1
21        outro_val = outro.valor if isinstance(outro, ContaDecimal) else Decimal(outro)
22        return ContaDecimal(func(self.valor, outro_val))
23
24    def __mul__(self, outro):
25        return self._bin_op(outro, lambda a, b: a * b)
26    __rmul__ = __mul__
27
```

```

28     def __add__(self, outro):
29         return self._bin_op(outro, lambda a, b: a + b)
30     __radd__ = __add__
31
32     def __sub__(self, outro):
33         return self._bin_op(outro, lambda a, b: a - b)
34
35     def __repr__(self):
36         return repr(self.valor)
37
38
39 def referencia_cosseno(x):
40     with localcontext() as ctx:
41         ctx.prec = ALTA
42
43         x = Decimal(x)
44         termo = Decimal("1")
45         soma = Decimal("1")
46         n = 1
47
48         for _ in range(50):
49             # Serie de Taylor para cosseno: 1 - x^2/2! + x^4/4! - ...
50             termo = -termo * x * x / Decimal((2*n - 1) * (2*n))
51             soma += termo
52
53             if abs(termo) < Decimal("1e-35"):
54                 break
55
56             n += 1
57
58         return soma
59
60
61 def coeficientes():
62     with localcontext() as ctx:
63         ctx.prec = K4
64         ctx.rounding = ROUND_HALF_EVEN
65
66         return (
67             Decimal("1") / Decimal("2"),
68             Decimal("1") / Decimal("24"),
69             Decimal("1") / Decimal("720"),
70             Decimal("1") / Decimal("40320")
71         )
72
73
74 A, B, C, D = coeficientes()
75
76
77 def nao_aninhado(z):
78     with localcontext() as ctx:
79         ctx.prec = K4
80         ctx.rounding = ROUND_HALF_EVEN
81
82         ContaDecimal.contador = 0
83         zc = ContaDecimal(z)
84         Ac, Bc, Cc, Dc = ContaDecimal(A), ContaDecimal(B), ContaDecimal(C), ContaDecimal(D)
85
86         z2 = zc * zc
87         z4 = z2 * z2
88         z6 = z4 * z2
89         z8 = z4 * z4
90
91         resultado = ContaDecimal(Decimal("1")) - z2 * Ac + z4 * Bc - z6 * Cc + z8 * Dc
92
93         return resultado.valor, ContaDecimal.contador
94
95
96 def aninhado(z):
97     with localcontext() as ctx:
98         ctx.prec = K4
99         ctx.rounding = ROUND_HALF_EVEN

```

```

100     ContaDecimal.contador = 0
101     y = ContaDecimal(z) * ContaDecimal(z)
102
103
104     # Coeficientes na ordem do maior grau (em y) ate o termo constante
105     coefs_horner = [D, -C, B, -A, Decimal("1")]
106
107     resultado = ContaDecimal(coefs_horner[0])
108     for c in coefs_horner[1:]:
109         resultado = resultado * y + ContaDecimal(c)
110
111     return resultado.valor, ContaDecimal.contador
112
113
114 if __name__ == "__main__":
115
116     print("QUESTAO 2")
117
118     print("\na)")
119     print("P8(x) = 1 - x^2/2 + x^4/24 - x^6/720 + x^8/40320")
120
121     with localcontext() as ctx:
122         ctx.prec = K4
123         ctx.rounding = ROUND_HALF_EVEN
124         z = Decimal("0.5")
125
126     print("\nArgumento usado:")
127     print("cos(0,5)")
128     print(f"z = {z}")
129
130     print("\nb)")
131     r_nao, op_nao = nao_aninhado(z)
132     print(f"Resultado nao aninhado = {r_nao}")
133     print(f"Operacoes = {op_nao}")
134
135     print("\nc)")
136     print("Com y = z^2:")
137     print("P8(z) = 1 + y(-1/2 + y(1/24 + y(-1/720 + y/40320)))")
138
139     print("\nd)")
140     r_ani, op_ani = aninhado(z)
141     print(f"Resultado aninhado = {r_ani}")
142     print(f"Operacoes = {op_ani}")
143
144     print("\ne)")
145     ref = referencia_cosseno("0.5")
146
147     erro_nao = abs(Decimal(str(r_nao)) - ref)
148     erro_ani = abs(Decimal(str(r_ani)) - ref)
149
150     print(f"Referencia cos(0,5) = {ref}")
151     print(f"Nao aninhado: {r_nao} | erro = {erro_nao:.4e} | {op_nao} ops")
152     print(f"Aninhado: {r_ani} | erro = {erro_ani:.4e} | {op_ani} ops")

```

Saída do Terminal

QUESTAO 2

a)

$$P8(x) = 1 - x^2/2 + x^4/24 - x^6/720 + x^8/40320$$

Argumento usado:

cos(0,5)

z = 0.5

b)

Resultado nao aninhado = 0.8776

Operacoes = 12

```

c)
Com y = z^2:
P8(z) = 1 + y(-1/2 + y(1/24 + y(-1/720 + y/40320)))

d)
Resultado aninhado = 0.8776
Operacoes = 9

e)
Referencia cos(0,5) = 0.8775825618903727161162815826038296519916
Nao aninhado: 0.8776 | erro = 1.7438e-5 | 12 ops
Aninhado:      0.8776 | erro = 1.7438e-5 | 9 ops

```

Fundamentação Matemática

A expansão de Maclaurin da função cosseno é dada por

$$\cos(x) = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}.$$

Considerando apenas os termos até o grau 8, obtém-se o polinômio

$$P_8(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!},$$

ou, substituindo os fatoriais,

$$P_8(x) = 1 - \frac{x^2}{2} + \frac{x^4}{24} - \frac{x^6}{720} + \frac{x^8}{40320}.$$

Como o enunciado pede diretamente uma aproximação para $\cos(0,5)$, o polinômio é avaliado em

$$z = 0,5,$$

sem necessidade de nenhuma identidade trigonométrica auxiliar. Como a atividade exige $k = 4$ dígitos, todas as operações utilizadas na avaliação do polinômio são realizadas com quatro algarismos significativos e arredondamento após cada operação.

Na avaliação sem aninhamento, calcula-se diretamente

$$P_8(z) = 1 - \frac{z^2}{2} + \frac{z^4}{24} - \frac{z^6}{720} + \frac{z^8}{40320}.$$

Considerando os coeficientes do polinômio previamente calculados, são necessárias quatro multiplicações para obter z^2 , z^4 , z^6 e z^8 , mais quatro multiplicações entre essas potências e os respectivos coeficientes e quatro somas ou subtrações. Dessa forma, são realizadas 12 operações aritméticas durante a avaliação.

Para obter a forma aninhada, define-se

$$y = z^2.$$

Assim,

$$P_8(z) = 1 - \frac{y}{2} + \frac{y^2}{24} - \frac{y^3}{720} + \frac{y^4}{40320}.$$

Aplicando o aninhamento semelhante ao método de Horner,

$$P_8(z) = 1 + y \left(-\frac{1}{2} + y \left(\frac{1}{24} + y \left(-\frac{1}{720} + y \frac{1}{40320} \right) \right) \right).$$

Nessa forma são necessárias uma multiplicação para calcular $y = z^2$, quatro multiplicações durante o processo de aninhamento e quatro somas ou subtrações, totalizando 9 operações.

Em vez de declarar essas quantidades como constantes fixas no código, ambas as funções (`nao_aninhado` e `aninhado`) utilizam um wrapper `ContaDecimal` que envolve o tipo `Decimal` e incrementa um contador

interno a cada multiplicação, soma ou subtração efetivamente executada. Dessa forma, o número de operações reportado é sempre calculado dinamicamente a partir do código executado, e não um valor fixado manualmente — o que garante que a contagem permaneça correta mesmo que o grau do polinômio ou a forma de avaliação sejam alterados no futuro.

Dessa forma, a forma aninhada reduz a quantidade de operações necessárias para avaliar o mesmo polinômio. Devido ao arredondamento realizado após cada operação, as duas formas poderiam, em princípio, gerar valores ligeiramente diferentes mesmo representando matematicamente o mesmo polinômio; no entanto, para $z = 0,5$ e $k = 4$, ambas convergem para o mesmo valor arredondado (0,8776) e, portanto, para o mesmo erro absoluto. Isso mostra que a vantagem da forma aninhada nem sempre se manifesta como uma redução no erro final, mas sim como uma redução no custo computacional (9 operações contra 12) — vantagem que se torna mais evidente quanto maior o grau do polinômio ou menor a precisão k utilizada.

Para a comparação do item (e), o valor de referência de $\cos(0,5)$ é calculado computacionalmente em precisão maior através de sua própria série de Maclaurin,

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots,$$

permitindo calcular o erro absoluto de cada aproximação por

$$E = |x_{\text{aproximado}} - x_{\text{referência}}|.$$

Questão 3: Lei de Resfriamento de Newton (Bisseção)

A temperatura inicial do dispositivo é $T_0 = 85^\circ C$, a temperatura ambiente é $T_a = 22^\circ C$ e a constante de decaimento é $k = 0.25$. Deseja-se encontrar o instante em que a temperatura atinge $35^\circ C$.

Código Python

```

1 def exponencial_manual(x, termos=50):
2     soma = 1.0
3     termo_atual = 1.0
4     for n in range(1, termos):
5         termo_atual = termo_atual * x / n
6         soma += termo_atual
7     return soma
8
9 def f(t):
10     return 63 * exponencial_manual(-0.25 * t) - 13
11
12 def bissecao(f, a, b, tol):
13     if f(a) * f(b) > 0:
14         return None
15
16     if f(a) == 0.0: return a
17     if f(b) == 0.0: return b
18
19     p_prev = a
20
21     while True:
22         p_n = (a + b) / 2.0
23
24         if abs(p_n - p_prev) < tol:
25             return p_n
26
27         if f(p_n) == 0.0:
28             return p_n
29
30         if f(a) * f(p_n) < 0:
31             b = p_n
32         else:
33             a = p_n

```

```

34         p_prev = p_n
35
36
37 def busca_intervalo_e_raiz(f, a_geral, b_geral, C, tol):
38     inicio = a_geral
39     while inicio < b_geral:
40         fim = min(inicio + C, b_geral)
41         if f(inicio) * f(fim) <= 0:
42             raiz = bissecao(f, inicio, fim, tol)
43             return (inicio, fim), raiz
44         inicio += C
45     return None, None
46
47 a, b, C, tol = 0, 30, 3, 1e-6
48 intervalo, raiz = busca_intervalo_e_raiz(f, a, b, C, tol)
49
50 if raiz is not None:
51     print(f"Subintervalo: [{intervalo[0]}, {intervalo[1]}]")
52     print(f"Tempo calculado (raiz t): {raiz:.6f}")
53     print(f"Verificacao: f({raiz:.6f}) = {f(raiz):.6e}")

```

Saída do Terminal

```

Subintervalo: [6, 9]
Tempo calculado (raiz t): 6.312742
Verificacao: f(6.312742) = -1.687198e-06

```

Fundamentação Matemática

A Lei de Resfriamento de Newton é dada por

$$T(t) = T_a + (T_0 - T_a)e^{-kt}.$$

Substituindo os valores do problema ($T_a = 22$, $T_0 = 85$, $k = 0,25$) e impondo a condição $T(t) = 35$, obtém-se

$$22 + (85 - 22)e^{-0,25t} = 35 \implies 63e^{-0,25t} = 13.$$

Reorganizando na forma $f(t) = 0$, como pede o item (a),

$$f(t) = 63e^{-0,25t} - 13 = 0.$$

Como o enunciado proíbe o uso de bibliotecas numéricas, a exponencial e^x é aproximada pela própria série de Maclaurin,

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!},$$

implementada de forma recursiva através da relação entre termos consecutivos $t_n = t_{n-1} \cdot \frac{x}{n}$, o que evita recalcular potências e fatoriais do zero a cada termo. Para os valores de $x = -0,25t$ utilizados aqui ($t \in [0, 30]$, logo $x \in [-7,5, 0]$), a série é alternada e converge rapidamente, tornando 50 termos mais que suficientes para atingir a precisão de ponto flutuante de dupla precisão.

A função $f(t) = 63e^{-0,25t} - 13$ é estritamente decrescente (pois $e^{-0,25t}$ é decrescente e $63 > 0$), o que garante que, caso exista uma raiz em um intervalo onde f troca de sinal, essa raiz é única nesse intervalo. Em $t = 0$, $f(0) = 63 - 13 = 50 > 0$; em $t = 30$, $f(30) \approx -12,97 < 0$. Pelo Teorema do Valor Intermediário, como f é contínua e $f(0) \cdot f(30) < 0$, existe pelo menos uma raiz em $(0, 30)$.

O item (c) automatiza a busca por essa mudança de sinal: o intervalo $[a, b] = [0, 30]$ é percorrido em subintervalos de comprimento $C = 3$, testando em cada um se $f(\text{início}) \cdot f(\text{fim}) \leq 0$. Ao encontrar o primeiro subintervalo com essa propriedade, o Método da Bisseção é aplicado dentro dele; caso nenhum subintervalo satisfaça a condição até o fim do intervalo geral, o algoritmo retorna nulo, evitando aplicar a Bisseção onde a existência da raiz não pode ser garantida.

Dentro do Método da Bisseção, a cada iteração calcula-se o ponto médio $p_n = \frac{a_n + b_n}{2}$ e mantém-se a metade do intervalo em que a mudança de sinal persiste. Como o comprimento do intervalo é reduzido pela metade a cada iteração, o erro após n iterações é limitado por

$$|p_n - t^*| \leq \frac{b - a}{2^n},$$

garantindo convergência para a raiz exata t^* independentemente do ponto de partida, desde que a condição inicial de troca de sinal seja satisfeita. O critério de parada adotado, $|p_n - p_{n-1}| < 10^{-6}$, interrompe o processo quando essa diferença, que decresce geometricamente, cai abaixo da tolerância especificada.

Aplicando os algoritmos com $a = 0$, $b = 30$ e $C = 3$, o subintervalo com mudança de sinal encontrado é $[6, 9]$, e a raiz obtida é $t \approx 6,312742$ minutos, o que é consistente com a solução analítica exata do problema,

$$t^* = -\frac{1}{0,25} \ln\left(\frac{13}{63}\right) \approx 6,312741 \text{ minutos,}$$

confirmando a precisão da abordagem numérica implementada.

Questão 4: Gráficos e Sequência de Aproximações da Bisseção

Código Python

```

1
2 import matplotlib.pyplot as plt
3
4
5 # a) Funcao que amostra e plota y = f(x) em [a, b] usando segmentos de reta
6
7 def plotar_funcao(f, a, b, n, ax=None, titulo="Grafico de f(x)":
8     """
9     Amostra f uniformemente em [a, b] com n pontos e plota y=f(x)
10    usando segmentos de reta ligando os pontos amostrados.
11
12    Parametros:
13    f: funcao a ser plotada
14    a, b: extremos do intervalo
15    n: quantidade de pontos amostrados uniformemente
16    ax: eixo matplotlib onde desenhar (cria um novo se None)
17    titulo: titulo do grafico
18
19    Retorna:
20    (xs, ys) -- listas com os pontos amostrados (uteis para reuso)
21    """
22    if ax is None:
23        _, ax = plt.subplots(figsize=(7, 5))
24
25    passo = (b - a) / (n - 1)
26    xs = [a + i * passo for i in range(n)]
27    ys = [f(x) for x in xs]
28
29    # desenha manualmente os segmentos de reta ligando (x_i, y_i) a (x_{i+1}, y_{i+1})
30    for i in range(n - 1):
31        ax.plot([xs[i], xs[i + 1]], [ys[i], ys[i + 1]], color="tab:blue", linewidth=1.5)
32
33    ax.axhline(0, color="black", linewidth=0.7)
34    ax.set_title(titulo)
35    ax.set_xlabel("x")
36    ax.set_ylabel("f(x)")
37    ax.grid(True, alpha=0.3)
38
39    return xs, ys
40
41
42 def funcao_teste(x):
43     """Funcao qualquer usada apenas para validar plotar_funcao(letra a)."""
44     # sin(x) usando serie de Maclaurin
45     sin_x = 0
46     termo = x
47     for n in range(1, 21):
48         sin_x += termo
49         termo = termo * (-(x**2)) / ((2*n) * (2*n + 1))
50     return sin_x + 0.3 * x

```

```

51
52
53
54 # b) Bissecao que retorna a sequencia (p_n) de aproximacoes, nao so a raiz
55
56 def bissecao_sequencia(f, a, b, tol, n_max=1000):
57     """
58     Igual ao Metodo da Bissecao da questao 3b, mas retorna a LISTA completa
59     de aproximacoes p_n geradas ao longo das iteracoes, em vez de apenas a
60     raiz final.
61
62     Retorna:
63     lista_p -- lista [p_1, p_2, ..., p_k] com todas as aproximacoes
64     de aproximacoes p_n geradas ao longo das iteracoes, em vez de apenas a
65     raiz final.
66     """
67     fa = f(a)
68     fb = f(b)
69
70     if fa * fb > 0:
71         print("f(a) e f(b) tem o mesmo sinal: nao ha garantia de raiz em [a, b].")
72         return []
73
74     lista_p = []
75     p_anterior = a
76     for _ in range(n_max):
77         p = (a + b) / 2
78         fp = f(p)
79         lista_p.append(p)
80
81         if fp == 0 or abs(p - p_anterior) < tol:
82             break
83
84         if fa * fp < 0:
85             b = p
86             fb = fp
87         else:
88             a = p
89             fa = fp
90
91         p_anterior = p
92
93     return lista_p
94
95 def plotar_convergencia(f, a, b, n, lista_p, titulo="f(x) e sequencia de aproximacoes"):
96     """
97     Plota y = f(x) (usando plotar_funcao) junto com os pontos
98     (p_n, f(p_n)) gerados pela bissecao, na ordem em que foram calculados.
99     """
100     fig, ax = plt.subplots(figsize=(7, 5))
101     plotar_funcao(f, a, b, n, ax=ax, titulo=titulo)
102
103     xs_p = lista_p
104     ys_p = [f(p) for p in lista_p]
105
106     ax.plot(xs_p, ys_p, "o--", color="tab:red", markersize=5,
107             linewidth=1, label="p_n (bissecao)")
108
109     for i, (x, y) in enumerate(zip(xs_p, ys_p)):
110         ax.annotate(str(i + 1), (x, y), textcoords="offsetpoints",
111                    xytext=(4, 4), fontsize=7, color="tab:red")
112
113     ax.legend()
114     return fig
115
116 # Execucao
117
118 if __name__ == "__main__":
119     # a) validacao com uma funcao qualquer
120     fig_a, ax_a = plt.subplots(figsize=(7, 5))
121     plotar_funcao(funcao_teste, a=0, b=10, n=200, ax=ax_a,
122                  titulo="Validacao da letra a): f(x) = sin(x) + 0.3x")
123     fig_a.savefig("List1/questao4a_validacao.png", dpi=150, bbox_inches="tight")

```

```

124     print("Grafico da letra a) salvo em questao4a_validacao.png")
125
126     # b) aplicacao em f(x) = ln(x) - 2^x + x^2 - 1, [3, 5]
127     def f_b(x):
128         # math.log(x) usando serie equivalente (Maclaurin para ln((1+y)/(1-y)))
129         y = (x - 1) / (x + 1)
130         log_x = 0
131         for n in range(50):
132             log_x += (y**(2*n + 1)) / (2*n + 1)
133         log_x *= 2
134         return log_x - 2 ** x + x ** 2 - 1
135
136     tol = 1e-6
137     lista_p = bissecao_sequencia(f_b, a=3, b=5, tol=tol)
138
139     print("=" * 55)
140     print("f(x) = ln(x) - 2^x + x^2 - 1, intervalo [3, 5]")
141     print("=" * 55)
142     for i, p in enumerate(lista_p, start=1):
143         print(f"p{i:<3d} = {p:.10f} f(p{i}) = {f_b(p):.6e}")
144     print("-" * 55)
145     print(f"Raiz aproximada: {lista_p[-1]:.10f}")
146     print(f"Numero de iteracoes: {len(lista_p)}")
147     print("=" * 55)
148
149     fig_b = plotar_convergencia(f_b, a=3, b=5, n=200, lista_p=lista_p,
150                                titulo=f"f(x) = ln(x) - 2^x + x^2 - 1 e sequencia p_n")
151
152     fig_b.savefig("List 1/questao4b_convergencia.png", dpi=150, bbox_inches="tight")
153     print("Grafico da letra b) salvo em questao4b_convergencia.png")
154
155     # -----
156     # Resultados numericos obtidos (saida da execucao acima):
157     #
158     # f(x) = ln(x) - 2^x + x^2 - 1, intervalo [3, 5]
159     #
160     # p1 = 4.0000000000 f(p1) = 3.862944e-01
161     # p2 = 4.5000000000 f(p2) = -1.873340e+00
162     # p3 = 4.2500000000 f(p3) = -5.178949e-01
163     # p4 = 4.1250000000 f(p4) = -1.543270e-02
164     # p5 = 4.0625000000 f(p5) = 1.973243e-01
165     # p6 = 4.0937500000 f(p6) = 9.400407e-02
166     # p7 = 4.1093750000 f(p7) = 4.006108e-02
167     # p8 = 4.1171875000 f(p8) = 1.250940e-02
168     # p9 = 4.1210937500 f(p9) = -1.412677e-03
169     # p10 = 4.1191406250 f(p10) = 5.560583e-03
170     # p11 = 4.1201171875 f(p11) = 2.077011e-03
171     # p12 = 4.1206054688 f(p12) = 3.329321e-04
172     # p13 = 4.1208496094 f(p13) = -5.396811e-04
173     # p14 = 4.1207275391 f(p14) = -1.033267e-04
174     # p15 = 4.120665039 f(p15) = 1.148147e-04
175     # p16 = 4.1206970215 f(p16) = 5.746984e-06
176     # p17 = 4.1207122803 f(p17) = -4.878910e-05
177     # p18 = 4.1207046509 f(p18) = -2.152087e-05
178     # p19 = 4.1207008362 f(p19) = -7.886898e-06
179     # p20 = 4.1206989288 f(p20) = -1.069945e-06
180     # p21 = 4.1206979752 f(p21) = 2.338523e-06
181     #
182     # Raiz aproximada: 4.1206979752
183     # Numero de iteracoes: 21
184     # (Graficos salvos em questao4a_validacao.png e questao4b_convergencia.png)

```

Saída do Terminal

Grafico da letra a) salvo em questao4a_validacao.png

f(x) = ln(x) - 2^x + x^2 - 1, intervalo [3, 5]

```

p1 = 4.0000000000 f(p1) = 3.862944e-01
p2 = 4.5000000000 f(p2) = -1.873340e+00

```

```

p3  = 4.2500000000    f(p3) = -5.178949e-01
p4  = 4.1250000000    f(p4) = -1.543270e-02
p5  = 4.0625000000    f(p5) = 1.973243e-01
p6  = 4.0937500000    f(p6) = 9.400407e-02
p7  = 4.1093750000    f(p7) = 4.006108e-02
p8  = 4.1171875000    f(p8) = 1.250940e-02
p9  = 4.1210937500    f(p9) = -1.412677e-03
p10 = 4.1191406250    f(p10) = 5.560583e-03
p11 = 4.1201171875    f(p11) = 2.077011e-03
p12 = 4.1206054688    f(p12) = 3.329321e-04
p13 = 4.1208496094    f(p13) = -5.396811e-04
p14 = 4.1207275391    f(p14) = -1.033267e-04
p15 = 4.1206665039    f(p15) = 1.148147e-04
p16 = 4.1206970215    f(p16) = 5.746984e-06
p17 = 4.1207122803    f(p17) = -4.878910e-05
p18 = 4.1207046509    f(p18) = -2.152087e-05
p19 = 4.1207008362    f(p19) = -7.886898e-06
p20 = 4.1206989288    f(p20) = -1.069945e-06
p21 = 4.1206979752    f(p21) = 2.338523e-06

```

Raiz aproximada : 4.1206979752

Numero de iteracoes: 21

=====

Grafico da letra b) salvo em questao4b_convergencia.png

Gráficos

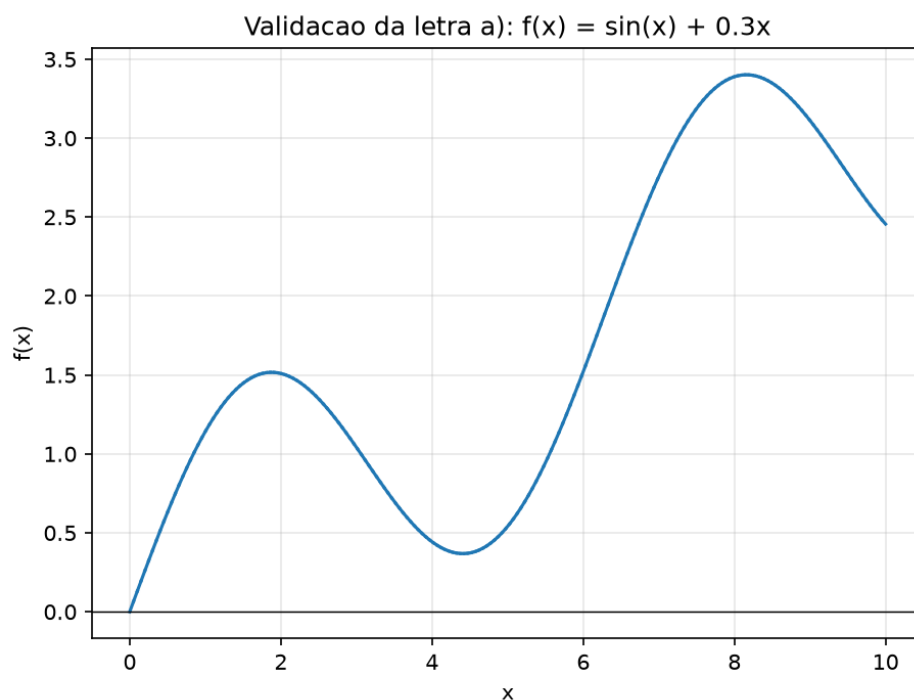


Figura 1: Validação da função de plotagem da letra (a).

Fundamentação Matemática

Na letra (a), o intervalo $[a, b]$ é dividido uniformemente em n pontos. O espaçamento entre pontos consecutivos é dado por

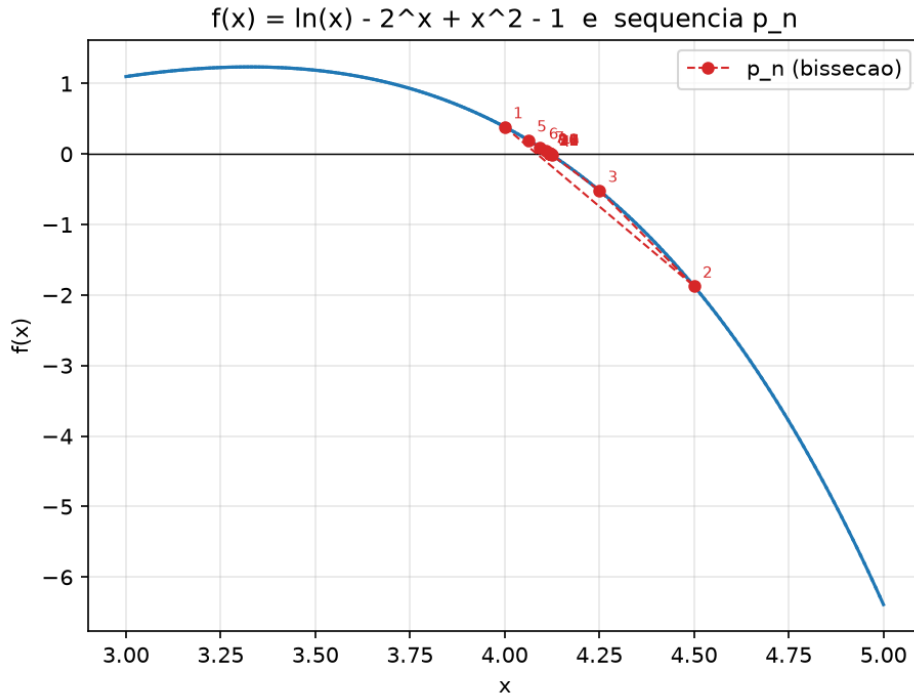


Figura 2: Função $f(x)$ e sequência de aproximações p_n obtida pelo Método da Bissecção.

$$h = \frac{b - a}{n - 1}.$$

Dessa forma, cada ponto do domínio é calculado por

$$x_i = a + ih, \quad i = 0, 1, \dots, n - 1.$$

Para cada x_i , calcula-se

$$y_i = f(x_i).$$

O gráfico é então construído ligando cada par de pontos consecutivos

$$(x_i, f(x_i)) \quad \text{e} \quad (x_{i+1}, f(x_{i+1}))$$

através de segmentos de reta. Dessa maneira, não é necessário utilizar uma biblioteca numérica para gerar os pontos do domínio, pois a própria implementação realiza a amostragem uniforme.

Na letra (b), utiliza-se o mesmo princípio do Método da Bissecção desenvolvido anteriormente. Para um intervalo inicial $[a, b]$ em que

$$f(a)f(b) < 0,$$

existe pelo menos uma raiz no intervalo, considerando a continuidade da função.

A cada iteração é calculado o ponto médio

$$p_n = \frac{a_n + b_n}{2}.$$

O sinal de $f(p_n)$ determina qual metade do intervalo deve ser mantida. Se

$$f(a_n)f(p_n) < 0,$$

o novo intervalo será

$$[a_n, p_n].$$

Caso contrário, será

$$[p_n, b_n].$$

A alteração em relação ao algoritmo original de Bissecção é que cada aproximação p_n é armazenada em uma lista. Assim, ao final do processo, em vez de retornar somente a aproximação final da raiz, o algoritmo fornece toda a sequência

$$p_1, p_2, p_3, \dots, p_n.$$

O critério de parada utilizado é a diferença entre duas aproximações consecutivas,

$$|p_n - p_{n-1}| < \varepsilon.$$

No experimento solicitado, aplica-se o método à função

$$f(x) = \ln(x) - 2^x + x^2 - 1$$

no intervalo

$$[3, 5].$$

Cada aproximação da Bissecção é então representada graficamente pelo ponto

$$(p_n, f(p_n)).$$

A sequência desses pontos permite visualizar como as aproximações se aproximam progressivamente do ponto em que a curva intercepta o eixo x , isto é, da raiz da equação.

Questão 5: Iteração de Ponto Fixo

O problema analisa a equação $x^2 + x - 1 = 0$ a partir de duas funções de iteração: $g_1(x) = \frac{1}{1+x}$ e $g_2(x) = \frac{1}{x} - 1$.

Código Python

```
1 def g1(x):
2     return 1.0 / (1.0 + x)
3
4 def g2(x):
5     return (1.0 / x) - 1.0
6
7 def ponto_fixo(g, x0, tol, Nmax):
8     x_old = x0
9     for i in range(1, Nmax + 1):
10         try:
11             x_new = g(x_old)
12         except ZeroDivisionError:
13             return None, i, "Erro: Divis o por zero."
14
15         if abs(x_new - x_old) < tol:
16             return x_new, i, "Sucesso (Convergiu)"
17
18         x_old = x_new
19
20     return None, Nmax, "Falha (Atingiu Nmax)"
21
22 x0, tol, Nmax = 0.5, 1e-6, 100
23
24 print("-> Testando g1(x)")
25 raiz_g1, iter_g1, status_g1 = ponto_fixo(g1, x0, tol, Nmax)
26 print(f"Status: {status_g1} | Raiz: {raiz_g1:.6f} | Iteracoes: {iter_g1}")
27
28 print("\n-> Testando g2(x)")
```

```

29 | raiz_g2, iter_g2, status_g2 = ponto_fixo(g2, x0, tol, Nmax)
30 | print(f"Status: {status_g2} | Iteracoes: {iter_g2}")

```

Saída do Terminal

-> Testando g1(x)

Status: Sucesso (Convergiu) | Raiz: 0.618034 | Iteracoes: 14

-> Testando g2(x)

Status: Erro: Divisão por zero. | Iteracoes: 3

Fundamentação Matemática

A diferença no comportamento decorre do Teorema do Ponto Fixo. Para uma função de iteração g contínua e diferenciável, se x^* é o ponto fixo (isto é, $g(x^*) = x^*$) e a sequência $x_{n+1} = g(x_n)$ está suficientemente próxima de x^* , o processo converge localmente quando $|g'(x^*)| < 1$ e diverge (ou oscila) quando $|g'(x^*)| > 1$.

Calculando as derivadas das duas formulações,

$$g_1'(x) = -\frac{1}{(1+x)^2} \quad \text{e} \quad g_2'(x) = -\frac{1}{x^2},$$

e avaliando-as no ponto fixo comum $x^* = \frac{\sqrt{5}-1}{2} \approx 0,618034$ (raiz positiva de $x^2 + x - 1 = 0$, dentro do intervalo $[0, 1]$ analisado), obtém-se

$$|g_1'(x^*)| = \frac{1}{(1+0,618034)^2} \approx 0,382 < 1,$$

garantindo, pelo Teorema do Ponto Fixo, a convergência local da iteração baseada em g_1 . Em contrapartida,

$$|g_2'(x^*)| = \frac{1}{(0,618034)^2} \approx 2,618 > 1,$$

caracterizando um processo iterativo instável: pequenos desvios em relação a x^* são amplificados a cada iteração.

Diferentemente de uma formulação em que a divergência se manifestaria apenas pelo não cumprimento do critério de parada até N_{max} iterações, a formulação $g_2(x) = \frac{1}{x} - 1$ evidencia essa instabilidade de forma direta e imediata. Como $x_0 = 0,5$, $x_1 = 1,0$ e $x_2 = 0,0$ são todos exatamente representáveis em ponto flutuante binário, a sequência de iterações cai exatamente sobre $x = 0$ — ponto em que g_2 não está definida — produzindo um erro de divisão por zero já na terceira iteração. Explicitando o rastreo:

$$x_1 = g_2(0,5) = \frac{1}{0,5} - 1 = 2 - 1 = 1,0,$$

$$x_2 = g_2(1,0) = \frac{1}{1,0} - 1 = 0,0,$$

$$x_3 = g_2(0,0) \rightarrow \text{indefinido (divisão por zero)}.$$

Esse comportamento é a manifestação empírica exata da instabilidade prevista teoricamente: como $|g_2'(x)| > 1$ ao longo de quase todo o intervalo $[0, 1]$ e a derivada diverge quando $x \rightarrow 0$, a iteração é repelida do ponto fixo em vez de atraída para ele, sendo empurrada rapidamente para fora do domínio de definição da função. O tratamento de exceção implementado (captura de `ZeroDivisionError`) garante que o algoritmo retorne de forma controlada nesse cenário, atendendo ao espírito do enunciado de retornar nulo quando não for possível encontrar a raiz.

Vale notar que $g_1'(x)$ satisfaz $|g_1'(x)| < 1$ para todo $x \in (0, 1]$, com $|g_1'(0)| = 1$ exatamente no extremo $x = 0$ — o que não compromete a convergência observada, já que a iteração parte de $x_0 = 0,5$ e permanece sempre em uma vizinhança de x^* distante desse extremo.

Assim, entre as duas formulações de ponto fixo para a mesma equação $x^2 + x - 1 = 0$, $g_1(x) = \frac{1}{1+x}$ é a mais adequada computacionalmente, pois sua derivada tem magnitude menor que 1 na região relevante

do intervalo $[0, 1]$, enquanto $g_2(x) = \frac{1}{x} - 1$ não satisfaz essa condição, tornando-a inadequada para a Iteração de Ponto Fixo neste problema.

Questão 6: Métodos de Newton e da Bisseção

Considere a equação

$$x^3 - 2x - 5 = 0.$$

Código Python

```

1
2
3 def f(x):
4     """f(x) = x^3 - 2x - 5"""
5     return x**3 - 2*x - 5
6
7
8 def df(x):
9     """f'(x) = 3x^2 - 2"""
10    return 3*x**2 - 2
11
12
13 # -----
14 # a) Metodo de Newton
15 # -----
16 def metodo_newton(f, df, x0, tol, n_max=100):
17     """
18     Metodo de Newton-Raphson para encontrar raiz de f(x) = 0.
19
20     Parametros:
21     f: funcao cuja raiz se deseja encontrar
22     df: derivada de f
23     x0: aproximacao inicial
24     tol: tolerancia (criterio de parada: |x_{n+1} - x_n| < tol)
25     n_max: numero maximo de iteracoes
26
27     Retorna:
28     (raiz, numero_de_iteracoes) ou (None, numero_de_iteracoes) se nao convergir
29     """
30     x_atual = x0
31     for i in range(1, n_max + 1):
32         fx = f(x_atual)
33         dfx = df(x_atual)
34
35         if dfx == 0:
36             print("Derivada nula: metodo de Newton falhou.")
37             return None, i
38
39         x_prox = x_atual - fx / dfx
40
41         if abs(x_prox - x_atual) < tol:
42             return x_prox, i
43
44         x_atual = x_prox
45
46     return None, n_max
47
48
49 # -----
50 # c) Metodo da Bissecao
51 # -----
52 def metodo_bissecao(f, a, b, tol, n_max=1000):
53     """
54     Metodo da Bissecao para encontrar raiz de f(x) = 0 em [a, b].
55 
```

```

56 Parametros:
57 f--funcao cuja raiz se deseja encontrar
58 a,b--extremos do intervalo (f(a) e f(b) devem ter sinais opostos)
59 tol--tolerancia (criterio de parada: |p_n - p_{n-1}| < tol)
60 n_max--numero maximo de iteracoes
61
62 Retorna:
63 (raiz, numero_de_iteracoes) ou (None, numero_de_iteracoes) se nao for
64 possivel encontrar a raiz
65 """
66 fa = f(a)
67 fb = f(b)
68
69 if fa * fb > 0:
70     print("f(a) e f(b) tem o mesmo sinal: nao ha garantia de raiz em [a, b].")
71     return None, 0
72
73 p_anterior = a
74 for i in range(1, n_max + 1):
75     p = (a + b) / 2
76     fp = f(p)
77
78     if fp == 0 or abs(p - p_anterior) < tol:
79         return p, i
80
81     if fa * fp < 0:
82         b = p
83         fb = fp
84     else:
85         a = p
86         fa = fp
87
88     p_anterior = p
89
90 return None, n_max
91
92
93 # -----
94 # a) Implementacao do Metodo de Newton
95 # A funcao metodo_newton() acima implementa o algoritmo solicitado.
96 #
97 # b) Aplicacao do Metodo de Newton:
98 # x0 = 2 e tolerancia = 10^-6
99 #
100 # c) Aplicacao do Metodo da Bissecao:
101 # intervalo [2, 3] e tolerancia = 10^-6
102 #
103 # d) Comparacao entre as raizes obtidas e o numero de iteracoes
104 # -----
105 if __name__ == "__main__":
106     tol = 1e-6
107
108     # b) Newton com x0 = 2
109     raiz_newton, iter_newton = metodo_newton(f, df, x0=2, tol=tol)
110
111     # c) Bissecao no intervalo [2, 3]
112     raiz_bissecao, iter_bissecao = metodo_bissecao(f, a=2, b=3, tol=tol)
113
114 print("=" * 55)
115 print("Questao 6 - x^3 - 2x - 5 = 0")
116 print("=" * 55)
117
118 print("a) Metodo de Newton implementado em metodo_newton().")
119
120 print("-" * 55)
121 print("b) Metodo de Newton (x0=2, tol=1e-6):")
122 print(f"raiz={raiz_newton:.10f}")
123 print(f"f(raiz)={f(raiz_newton):.2e}")
124 print(f"iteracoes={iter_newton}")
125
126 print("-" * 55)
127 print("c) Metodo da Bissecao ([2, 3], tol=1e-6):")
128 print(f"raiz={raiz_bissecao:.10f}")

```

```

129 print(f"_{f(raiz)}_{f(raiz_bissecao):.2e}")
130 print(f"_{iteracoes}_{iter_bissecao}")
131
132 print("-" * 55)
133 print("d)_Comparacao:")
134 print(f"_{Diferenca_entre_raizes}_="
135       f"{abs(raiz_newton-_raiz_bissecao):.2e}")
136 print(f"_{Newton}_{iter_newton}_{iteracoes}")
137 print(f"_{Bissecao}_{iter_bissecao}_{iteracoes}")
138 print(f"_{A_Bissecao_utilizou}")
139 print(f"_{iter_bissecao-_iter_newton}_{iteracoes}_a_mais.")
140 print("-" * 55)
141
142 # -----
143 # Resultados numericos obtidos (saida da execucao acima):
144 #
145 # =====
146 # Equacao: x^3 - 2x - 5 = 0
147 # =====
148 # Metodo de Newton (x0 = 2):
149 #   raiz          = 2.0945514815
150 #   f(raiz)       = -8.88e-16
151 #   iteracoes     = 4
152 # -----
153 # Metodo da Bissecao ([2, 3]):
154 #   raiz          = 2.0945520401
155 #   f(raiz)       = 6.23e-06
156 #   iteracoes     = 20
157 # -----
158 # Diferenca entre as raizes: 5.59e-07
159 # Newton precisou de 4 iteracoes.
160 # Bissecao precisou de 20 iteracoes (16 a mais que Newton).
161 # =====

```

Saída do Terminal

```

=====
Questao 6 - x^3 - 2x - 5 = 0
=====
a) Metodo de Newton implementado em metodo_newton().
-----
b) Metodo de Newton (x0 = 2, tol = 1e-6):
   raiz          = 2.0945514815
   f(raiz)       = -8.88e-16
   iteracoes     = 4
-----
c) Metodo da Bissecao ([2, 3], tol = 1e-6):
   raiz          = 2.0945520401
   f(raiz)       = 6.23e-06
   iteracoes     = 20
-----
d) Comparacao:
   Diferenca entre as raizes = 5.59e-07
   Newton:    4 iteracoes
   Bissecao: 20 iteracoes
   A Bissecao utilizou 16 iteracoes a mais.
=====

```

Fundamentação Matemática

Na letra (a), aplica-se o Método de Newton à função

$$f(x) = x^3 - 2x - 5.$$

Sua derivada é

$$f'(x) = 3x^2 - 2.$$

O Método de Newton constrói uma sequência de aproximações utilizando

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Para a função do problema, a expressão utilizada no algoritmo é

$$x_{n+1} = x_n - \frac{x_n^3 - 2x_n - 5}{3x_n^2 - 2}.$$

Na letra (b), a aproximação inicial determinada pelo enunciado é

$$x_0 = 2,$$

e o processo iterativo é executado até que a diferença entre duas aproximações consecutivas satisfaça

$$|x_{n+1} - x_n| < 10^{-6}.$$

O Método de Newton utiliza informações sobre a derivada da função para determinar a próxima aproximação, o que geralmente permite uma convergência rápida quando a aproximação inicial está suficientemente próxima da raiz.

Na letra (c), aplica-se o Método da Bisseção no intervalo

$$[2, 3].$$

Nos extremos desse intervalo,

$$f(2) = 2^3 - 2(2) - 5 = -1$$

e

$$f(3) = 3^3 - 2(3) - 5 = 16.$$

Assim,

$$f(2)f(3) < 0,$$

o que garante, pela continuidade da função, a existência de pelo menos uma raiz no intervalo.

A cada iteração da Bisseção calcula-se

$$p_n = \frac{a_n + b_n}{2},$$

e verifica-se em qual metade do intervalo permanece a mudança de sinal. O processo é repetido até que a diferença entre aproximações consecutivas seja inferior à tolerância

$$10^{-6}.$$

Na letra (d), comparam-se as aproximações obtidas pelos dois métodos e a quantidade de iterações necessárias para atingir a tolerância especificada.

Embora ambos os algoritmos procurem a mesma raiz, possuem comportamentos distintos. A Bisseção reduz o intervalo aproximadamente pela metade a cada iteração e apresenta convergência garantida quando existe mudança de sinal no intervalo inicial. Já o Método de Newton utiliza o valor da derivada para produzir novas aproximações e, quando as condições são favoráveis, pode convergir em um número significativamente menor de iterações.

A comparação computacional permite observar essa diferença de eficiência através do número de iterações realizado por cada método.